



## **UNIVERSIDAD POLITÉCNICA DE CHIAPAS**

### **INGENIERIA EN TECNOLOGIAS DE LA INFORMACION E INNOVACION DIGITAL**

---

#### **MATERIA:**

Bases de Datos Avanzadas

#### **PROFESOR:**

Ramses Camas Najera

#### **ACTIVIDAD:**

Examen Parcial U1

#### **INTEGRANTES:**

Kelli Anahi Felipe Jacinto - 243762

Yasmin Afrodita Castellanos Peres - 243727

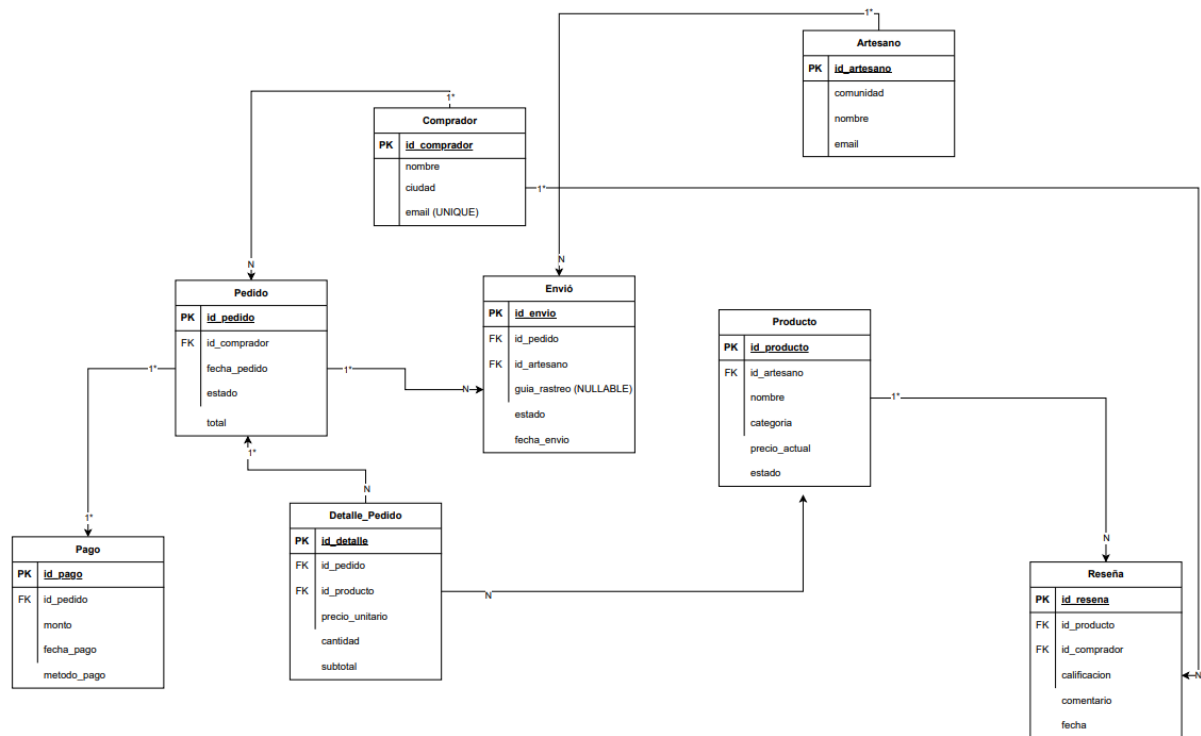
#### **5 CUATRIMESTRE**

**Grupo: 5°C**

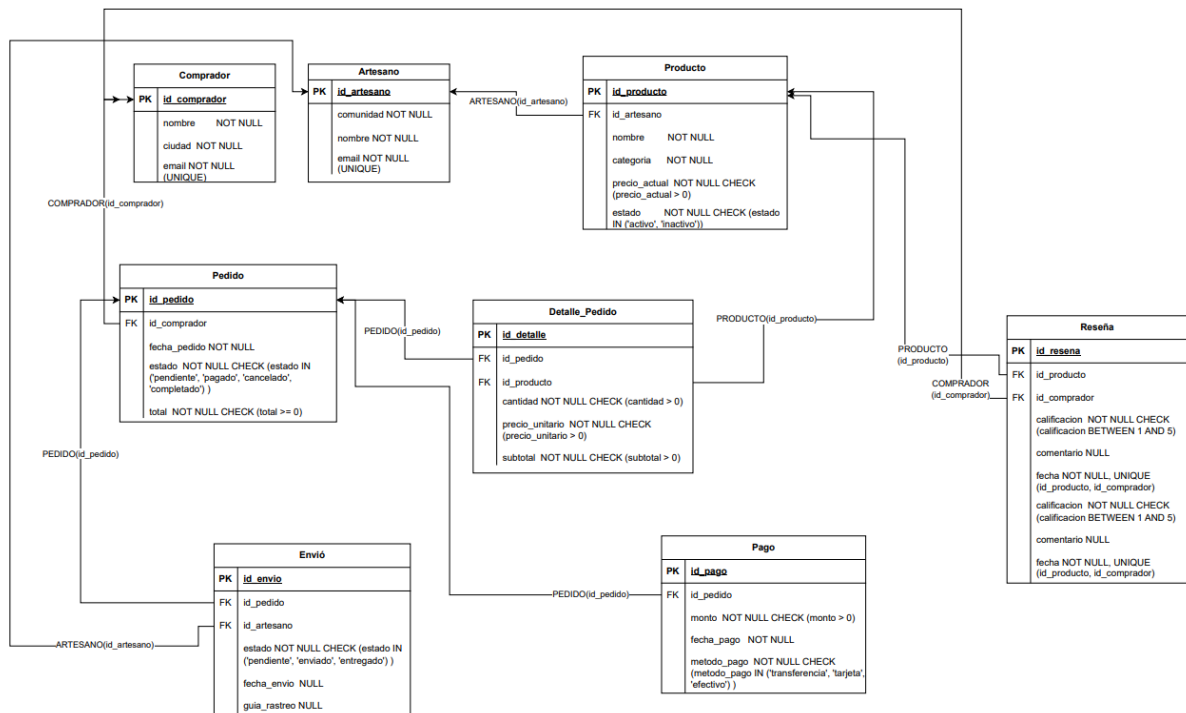
05 de febrero de 2026. Suchiapa, Chiapas

# 1. Modelo conceptual (ER) con cardinalidades y esquema relacional (tablas, PK, FK).

## Modelo conceptual



## Esquema relacional



## 2. Restricciones identificadas

### 1. UNIQUE - Reseña única por comprador y product

```

ALTER TABLE RESEÑA
ADD CONSTRAINT uq_resena_comprador_producto
UNIQUE (id_comprador, id_producto);
  
```

**Justificación:** María no puede reseñar el mismo collar dos veces.

### 2. CHECK - Calificación entre 1 y 5

```

ALTER TABLE RESEÑA
ADD CONSTRAINT chk_calificacion
CHECK (calificacion BETWEEN 1 AND 5);
  
```

**Justificación:** Las estrellas solo pueden ser de 1 a 5.

### 3. CHECK - Estado del product

```

ALTER TABLE PRODUCTO
  
```

```
ADD CONSTRAINT chk_estado_producto  
CHECK (estado IN ('activo', 'inactivo'));
```

***Justificación: Don Luis puede activar/desactivar productos (jarroncito).***

#### **4. CHECK - Estado del pedido**

```
ALTER TABLE PEDIDO  
ADD CONSTRAINT chk_estado_pedido  
CHECK (estado IN ('pendiente', 'pagado', 'completado', 'cancelado'));
```

***Justificación: El pedido de Lucía se canceló, el de María está completado.***

#### **5. CHECK - Precio positivo**

```
ALTER TABLE PRODUCTO  
ADD CONSTRAINT chk_precio_positivo  
CHECK (precio_actual > 0);
```

***Justificación: Los productos no pueden ser gratis o negativos.***

#### **6. NOT NULL - Campos obligatorios**

```
ALTER TABLE PRODUCTO  
ADD CONSTRAINT nn_precio  
CHECK (precio_actual IS NOT NULL);
```

```
ALTER TABLE PEDIDO  
ADD CONSTRAINT nn_fecha  
CHECK (fecha_pedido IS NOT NULL);
```

```
ALTER TABLE DETALLE_PEDIDO  
ADD CONSTRAINT nn_precio_compra  
CHECK (precio_unitario_compra IS NOT NULL);
```

***Justificación: Datos esenciales que siempre deben existir.***

#### **7. CHECK - Comisión del 12%**

```
-- Esta se podría implementar en una tabla de configuración  
-- o como columna calculada en reportes  
CHECK (comision = 0.12)
```

***Justificación: La plataforma siempre cobra 12% de comisión.***

#### **8. CHECK - Estado del envi**

```
ALTER TABLE ENVIO
```

```
ADD CONSTRAINT chk_estado_envio  
CHECK (estado IN ('preparando', 'enviado', 'entregado'));
```

**Justificación:** *Don Manuel ya envió, Doña Rosa está empacando.*

## 9. Integridad Referencial (FK)

```
-- Todas tus Foreign Keys son restricciones de integridad referencial  
ALTER TABLE PRODUCTO  
ADD CONSTRAINT fk_producto_artesano  
FOREIGN KEY (id_artesano) REFERENCES ARTESANO(id_artesano);  
  
ALTER TABLE PEDIDO  
ADD CONSTRAINT fk_pedido_comprador  
FOREIGN KEY (id_comprador) REFERENCES COMPRADOR(id_comprador);
```

y así con todas las FK

**Justificación:** *No puede haber productos huérfanos sin artesano, ni pedidos sin comprador.*

## 3. Queries SQL de ejemplo

1. Todos los productos activos con nombre del artesano y comunidad de origen

```
SELECT  
    p.nombre AS nombre_producto,  
    p.precio_actual,  
    a.nombre AS nombre_artesano,  
    a.comunidad_origen  
FROM PRODUCTO p  
INNER JOIN ARTESANO a ON p.id_artesano = a.id_artesano  
WHERE p.estado = 'activo';
```

**Explicación:** Usa INNER JOIN para unir productos con artesanos. Pedro busca productos activos y ve el jarroncito de Don Luis cuando lo reactiva.

2. Ingreso total por categoría de producto en un periodo dado

```
SELECT  
    p.categoria,  
    SUM(dp.subtotal) AS ingreso_total  
FROM DETALLE_PEDIDO dp  
INNER JOIN PRODUCTO p ON dp.id_producto = p.id_producto
```

```

INNER JOIN PEDIDO pd ON dp.id_pedido = pd.id_pedido
WHERE pd.estado IN ('pagado', 'completado')
  AND pd.fecha_pedido BETWEEN '2026-01-01' AND '2026-01-31'
GROUP BY p.categoria
ORDER BY ingreso_total DESC;

```

**Explicación:** Agrupa por categoría (ámbar, textil, cerámica) y suma los subtotales. Solo cuenta pedidos pagados o completados.

3. Pedidos donde al menos un envío sigue sin entregar

```

SELECT DISTINCT
  pd.id_pedido,
  pd.fecha_pedido,
  c.nombre AS comprador
FROM PEDIDO pd
INNER JOIN COMPRADOR c ON pd.id_comprador = c.id_comprador
INNER JOIN ENVIO e ON pd.id_pedido = e.id_pedido
WHERE e.estado IN ('preparando', 'enviado')
  AND pd.estado != 'cancelado';

```

**Explicación:** Busca pedidos con envíos que NO estén entregados. El pedido de María tiene el huipil de Doña Rosa pendiente (preparando).

4. Los compradores que más han gastado, con número de pedidos

```

SELECT
  c.nombre,
  c.email,
  COUNT(DISTINCT p.id_pedido) AS num_pedidos,
  SUM(p.total) AS gasto_total
FROM COMPRADOR c
INNER JOIN PEDIDO p ON c.id_comprador = p.id_comprador
WHERE p.estado IN ('pagado', 'completado')
GROUP BY c.id_comprador, c.nombre, c.email
ORDER BY gasto_total DESC
LIMIT 10;

```

**Explicación:** Agrupa por comprador, cuenta pedidos y suma totales. María aparecería con su pedido de 2,050 MXN.

5. Para cada artesano, proporción de productos activos vs inactivos

```

SELECT
  a.nombre,
  COUNT(CASE WHEN p.estado = 'activo' THEN 1 END) AS productos_activos,
  COUNT(CASE WHEN p.estado = 'inactivo' THEN 1 END) AS productos_inactivos,
  ROUND(
    COUNT(CASE WHEN p.estado = 'activo' THEN 1 END) * 100.0 /

```

```

COUNT(*), 2
) AS porcentaje_activos
FROM ARTESANO a
LEFT JOIN PRODUCTO p ON a.id_artesano = p.id_artesano
GROUP BY a.id_artesano, a.nombre
HAVING COUNT(p.id_producto) > 0;

```

**Explicación:** Usa CASE para contar activos e inactivos separadamente. Don Luis tendría el jarroncito cambiando entre activo/inactivo.

## 4. Reportes como VIEWS

### VIEW 1: Resumen de ventas por artesano (con comisión)

```

CREATE VIEW vista_ventas_artesano AS
SELECT
  a.id_artesano,
  a.nombre AS nombre_artesano,
  a.comunidad_origen,
  COUNT(DISTINCT dp.id_pedido) AS num_pedidos,
  SUM(dp.subtotal) AS total_vendido,
  ROUND(SUM(dp.subtotal) * 0.12, 2) AS comision_plataforma,
  ROUND(SUM(dp.subtotal) * 0.88, 2) AS pago_artesano
FROM ARTESANO a
INNER JOIN PRODUCTO p ON a.id_artesano = p.id_artesano
INNER JOIN DETALLE_PEDIDO dp ON p.id_producto = dp.id_producto
INNER JOIN PEDIDO pd ON dp.id_pedido = pd.id_pedido
WHERE pd.estado IN ('pagado', 'completado')
GROUP BY a.id_artesano, a.nombre, a.comunidad_origen;

```

### VIEW 2: Ranking de productos por calificación promedio

```

CREATE VIEW vista_ranking_productos AS
SELECT
  p.id_producto,
  p.nombre AS nombre_producto,
  p.categoria,

```

```

a.nombre AS nombre_artesano,
COUNT(r.id_resena) AS num_resenas,
ROUND(AVG(r.calificacion), 2) AS calificacion_promedio
FROM PRODUCTO p
INNER JOIN ARTESANO a ON p.id_artesano = a.id_artesano
INNER JOIN RESEÑA r ON p.id_producto = r.id_producto
GROUP BY p.id_producto, p.nombre, p.categoria, a.nombre
HAVING COUNT(r.id_resena) >= 3
ORDER BY calificacion_promedio DESC, num_resenas DESC;

```

### **VIEW 3: Reporte de pedidos rápidos vs tardíos**

```

CREATE VIEW vista_pedidos_velocidad_pago AS
SELECT
    pd.id_pedido,
    c.nombre AS comprador,
    pd.fecha_pedido,
    pg.fecha_pago,
    pd.total,
    ROUND(
        (JULIANDAY(pg.fecha_pago) - JULIANDAY(pd.fecha_pedido)) * 24,
        2
    ) AS horas_transcurridas,
    CASE
        WHEN JULIANDAY(pg.fecha_pago) - JULIANDAY(pd.fecha_pedido) <= 1
        THEN 'RÁPIDO'
        ELSE 'TARDÍO'
    END AS velocidad_pago
FROM PEDIDO pd
INNER JOIN COMPRADOR c ON pd.id_comprador = c.id_comprador
INNER JOIN PAGO pg ON pd.id_pedido = pg.id_pedido
WHERE pd.estado IN ('pagado', 'completado')
ORDER BY pd.fecha_pedido DESC;

```

## **5. Preguntas**

### **¿Por qué escogimos este caso?**

Escogimos el caso del Marketplace de Productos Artesanales porque presenta una complejidad adecuada para un equipo de dos integrantes y permite modelar múltiples situaciones reales de comercio electrónico.

Además, el caso permite aplicar conceptos clave de Bases de Datos Avanzadas como relaciones N–N, restricciones de integridad, VIEWS con agregaciones y decisiones de



diseño que deben justificarse, lo cual lo hace ideal para demostrar comprensión del modelado y no solo sintaxis SQL.

### **¿Por qué lo diseñamos así?**

El diseño prioriza la integridad de los datos y las reglas del negocio.

Se separó Pedido de Envío porque un pedido puede incluir productos de distintos artesanos, cada uno con su propio estado de envío.

La relación pedido–producto se resolvió con la tabla puente DETALLE\_PEDIDO para manejar la relación N–N y conservar el precio histórico de compra.

Las reseñas se modelaron como una entidad independiente con restricción UNIQUE para evitar calificaciones duplicadas.

Finalmente, se incluyó la entidad Pago para conservar el historial financiero de los pedidos, incluso cuando son cancelados.